

Mental model for designing a 3D array (this matters)

Forget NumPy syntax first. Think in **questions and axes**.

A **3D array** = **3 independent questions** answered together.

The core rule (non-negotiable)

Each dimension answers ONE question.

If you can't name the question, the dimension is wrong.

Step-by-step mental model

Step 1: Identify the entity (what repeats)

Ask:

"What thing am I collecting data for?"

Examples:

- City
- Student
- Machine
- User
- Image

This becomes **axis 0** (outermost).

Step 2: Identify the grouping inside it

Ask:

"Inside one entity, what repeats?"

Examples:

- Days inside a city
- Tests inside a student
- Sessions inside a user

This becomes **axis 1**.

Step 3: Identify the measurement depth

Ask:

"For one group, what multiple values exist?"

Examples:

- Hourly readings
- Subject-wise marks
- RGB channels of an image

This becomes **axis 2**.

Concrete example (real world)

Problem:

Temperature data for **cities** → **days** → **hourly readings**

Answer the questions:

1. Which city? → axis 0
2. Which day? → axis 1
3. Which hour? → axis 2

Shape:

(cities, days, hours)

Visual stack model (important)

Think like **files and folders**:

```
City 0
├─ Day 0 → [24 values]
├─ Day 1 → [24 values]
└─ Day 2 → [24 values]

City 1
├─ Day 0 → [24 values]
├─ Day 1 → [24 values]
└─ Day 2 → [24 values]
```

That is a 3D array. Not math. Storage.

Code example (only after thinking)

```
import numpy as np

temps = np.zeros((2, 3, 24))
```

Meaning:

- 2 cities
 - 3 days per city
 - 24 readings per day
-

Axis intuition (this prevents bugs)

- `axis=0` → across **entities**
- `axis=1` → across **groups inside entity**
- `axis=2` → across **measurements**

Example:

```
temps.mean(axis=2) # avg per day
temps.mean(axis=1) # avg per city
temps.mean(axis=0) # avg per day across cities
```

Design sanity check (use this every time)

Before creating a 3D array, say out loud:

"This array is indexed by [X][Y][Z]"

If you can't say that cleanly, redesign.

One-line rule to remember

Multidimensional arrays are not about dimensions. They are about answering multiple questions at once.

Once you think this way, NumPy becomes a data-modeling tool, not a math library.